

Wochenprotokoll: VisionQuest

Name: Matthias

Datum: 27.02.2026

Meilenstein: M2

Projektstatus:  Im Plan /  Verzögert /  Kritisch

1. Was wurde diese Woche gemacht?

- SQLite Datenbank mit 4 Tabellen (users, quests, user_stats, achievements) erstellt
- CRUD-Operationen für User und Quests implementiert
- 13 REST-API Endpoints definiert und getestet
- JWT-Authentication und Passwort-Hashing mit bcryptjs integriert
- Error-Handling für alle API-Endpoints implementiert
- Code-Struktur mit Service-Layer organisiert
- API-Dokumentation geschrieben (KI generiert mit Github CoPilot)

2. Wie liegt man im Zeitplan?

- Gut im Zeitplan – alle für 02.03.2026 geplanten Tasks abgeschlossen
- Backend läuft stabil mit SQLite und vollständiger Authentication

3. Was sind die nächsten Schritte?

- Flutter Frontend mit Login/Register Screens erweitern
- Alle API-Endpoints mit curl/Postman testen
- Token-Storage in Flutter implementieren
- Gamification-Features (XP, Levels, Streaks) testen
- npm Security Vulnerabilities beheben

4. KI-Einsatz & Dokumentation

Modell: GitHub Copilot (GPT-4)

1. Error-Handling für API und Datenbank

- **Prompt:** „Mach ein passendes Error-Handling“
- **Ergebnis:** Try-Catch-Blöcke um alle SQL-Queries, einheitliches Error-Response Format, verständliche Fehlermeldungen für den Client
- **Integration:** Die Blöcke wurden direkt in die CRUD-Funktionen der Endpunkte eingebaut. Fehler werden über die Middleware an den Client zurückgegeben, sodass sie konsistent und nachvollziehbar behandelt werden.

2. Input-Validierung

- **Prompt:** „Implementier eine Input-Validierung“
- **Ergebnis:** Validierungsfunktionen für alle POST/PUT Endpoints, ungültige oder unvollständige Eingaben werden erkannt, entsprechende Fehlermeldungen erzeugt
- **Integration:** Die Validierungen werden zu Beginn der Endpoint-Funktionen aufgerufen. Bei fehlerhaften Daten wird der Request abgebrochen und eine standardisierte Fehlermeldung zurückgegeben.

3. Verbesserung der Code-Struktur

- **Prompt:** „Sorge für eine bessere Code-Struktur in meinem Chaos“
- **Ergebnis:** CRUD-Funktionen ausgelagert in *dbService.js*, Funktionen nach logischen Bereichen (Users, Quests, Stats) gruppiert, Namenskonventionen vereinheitlicht, JSDoc-Kommentare ergänzt
- **Integration:** Endpunkte importieren die Funktionen aus *dbService.js*. Dadurch sind die API-Routen schlanker, übersichtlicher und leichter wartbar.

4. JWT-Token-Validierung

- **Prompt:** „Warum verifiziert mein JWT Token nicht richtig?“
- **Ergebnis:** Hinweise zum richtigen Secret-Key und Token-Decoding, damit Token korrekt verifiziert werden
- **Integration:** JWT-Middleware prüft jetzt alle geschützten Endpunkte, dekodiert Token korrekt und gibt aussagekräftige Fehlermeldungen bei Problemen zurück.

5. Doppelte Benutzerregistrierung verhindern

- **Prompt:** „Wie verhindere ich doppelte User-Registrierungen?“
- **Ergebnis:** Prüfung, ob die E-Mail-Adresse bereits existiert, und Rückgabe einer Fehlermeldung, falls sie schon registriert ist
- **Integration:** Die Prüfung erfolgt direkt vor der INSERT-Query in der Registrierungsfunktion. Somit werden doppelte Einträge vermieden und der Client erhält sofort Feedback.

6. Architektur & API-Design prüfen

- **Prompt:** „Checke mein SQLite-Datenbankschema auf Sicherheit, Struktur und Sinnhaftigkeit“ und „Wie sollte ich Error-Handling in einer REST API standardisieren?“
- **Ergebnis:** Validierung der Architektur, Optimierung des API-Designs, Verbesserung von Error-Handling und Code-Struktur, Planung für Testing und Dokumentation
- **Integration:** Ergebnisse flossen in die Endpunktstruktur, Error-Handling-Strategie und Code-Organisation ein, wodurch die Grundlage für sauberes Testing und Dokumentation geschaffen wurde.